

FC7xxx UART User Manual

Rev.A0

Revision History

Revision	Date	Changes
A0	2025/01/20	Add support for FC7300F8MDQ
A1	2025/12/12	Change Uart_AsyncReceive API

Table of Contents

Chapter 1 UART Introduction	5
1.1 Requirements	5
1.2 Design Summary	5
1.3 Hardware Summary	5
Chapter 2 Software Design	7
2.1 Rejected Requirements	7
2.2 File Structure	7
2.3 Macros	8
2.3.1 Macros in Uart_Types.h	8
2.3.2 Macros in Uart_Version.h	10
2.4 Enums	10
2.4.1 Enums in Uart_GeneralTypes.h	10
2.4.2 Enums in Uart_Types.h	12
2.5 Typedefs	12
2.5.1 Typedefs in Uart_Types.h	12
2.6 Structures	13
2.6.1 Uart_StateStructureType	13
2.6.2 Uart_UserConfigType	13
2.6.3 Uart_HwConfigType	14
2.6.4 Uart_ChannelConfigType	14
2.6.5 Uart_ConfigType	15
2.7 API Functions	15
2.7.1 Functions in CDD_Uart.h	15
2.8 Hardware Functions	21
2.8.1 Functions in LLD_Uart.h	21
2.9 Peripheral Functions	27
2.9.1 Functions in Uart_RegOps.h	27
2.10 API Sequence Diagram	35
2.10.1 Synchronous transmission	35
2.10.2 Asynchronous transmission	35
Chapter 3 Tresos Configuration Items	36
3.1 Containers and Variables	36
3.1.1 IMPLEMENTATION_CONFIG_VARIANT	36
3.1.2 GeneralConfiguration	36
3.1.3 UartEcucPartitionRef	38
3.1.4 UartGlobalConfig	38
3.1.5 ModuleConfiguration	39
3.1.6 CommonPublishedInformation	43
Chapter 4 Configuration Guides	46

4.1	UART Usage Common Steps.....	46
4.2	UART Channel Demo	46

Chapter 1 UART Introduction

1.1 Requirements

UART is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module. It has vendor-specific requirements and implementation.

1.2 Design Summary

The Uart driver is a part of the real-time driver that performs hardware access and provides a hardware-independent API to the application. The Uart driver is implemented as an Autosar complex device driver. It uses the Uart hardware peripheral and provides support for implementing the Uart protocol. The hardware and software settings can be configured using the Autosar standard configuration tool. The driver reports errors to the error manager defined in AUTOSAR. The Uart module of FC7xxx implements the following features:

- Full-duplex communication
- Configurable baud rate. It supports standard UART baud rates
- Configurable parity, stop bits
- Use in a multi-core environment
- DMA transfer
- Callback notification
- Continuous transfer in an asynchronous manner

1.3 Hardware Summary

Features of the FC Universal Asynchronous Receiver Transmitter (FCUART) module include:

- Full-duplex, standard non-return-to-zero (NRZ) format
- Independent FIFO structure for transmitting and receiving:
 - Both Transmit and Receive data FIFO depth are 16
 - Watermark for receiving and transmitting requests are separate configurable
 - Option for the receiver to assert request after a configurable number of idle characters if receive FIFO is not empty
- Programmable baud rate and oversampling ratio
- Use the receiver to monitor other FCUART transmitters
- Programmable data length from 7-bit to 10-bit
- Configurable transmit output and receive input polarity
- Multiple interrupt or DMA request source
- Programmable 1-bit or 2-bit stop bits
- Support operation in Stop mode; FCUART0 supports operation in standby mode
- Three receiver wakeup methods including address mark wakeup, idle line wakeup and receive data match
- Automatic address matching to reduce ISR overhead:
 - Address mark matching
 - Idle line address matching
 - Address alternate matching

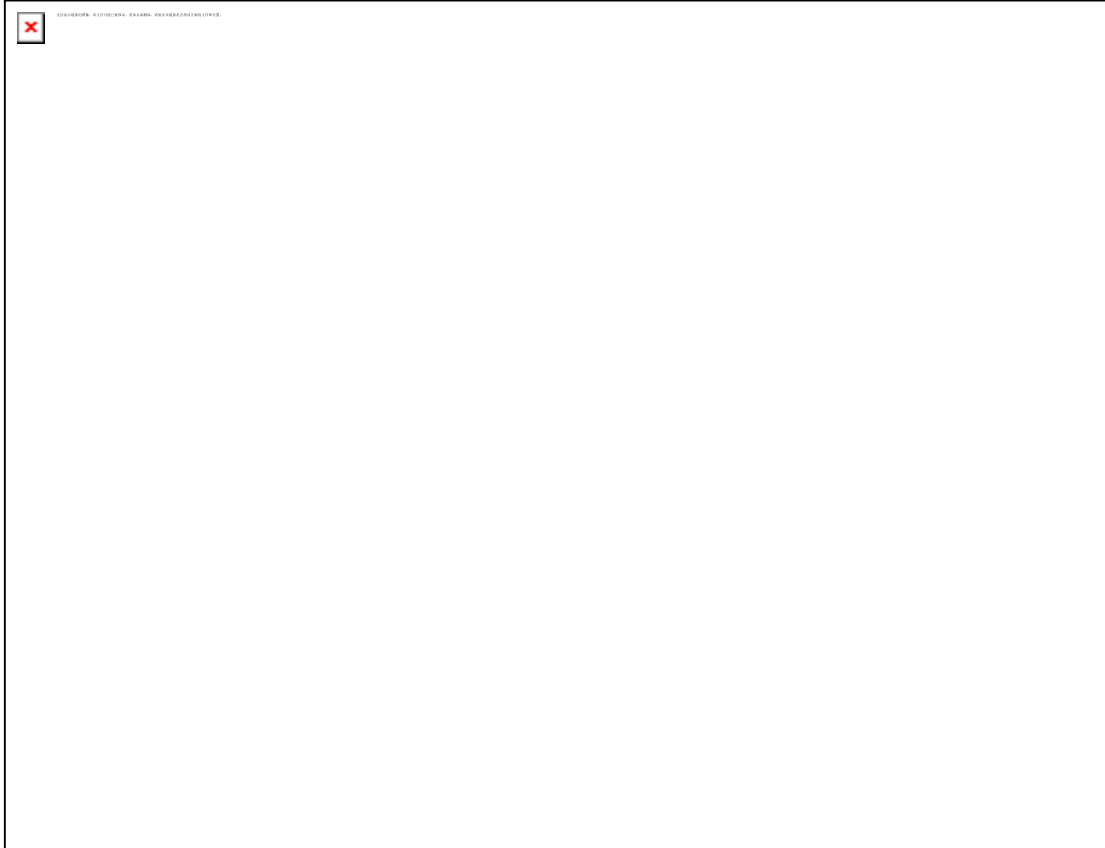
- Optional 13-bit break character generation / 11-bit break character detection
- Configurable idle length detection supporting 1, 2, 4, 8, 16, 32, 64 or 128 idle characters
- Request to send (RTS) and clear to send (CTS) hardware control flow support

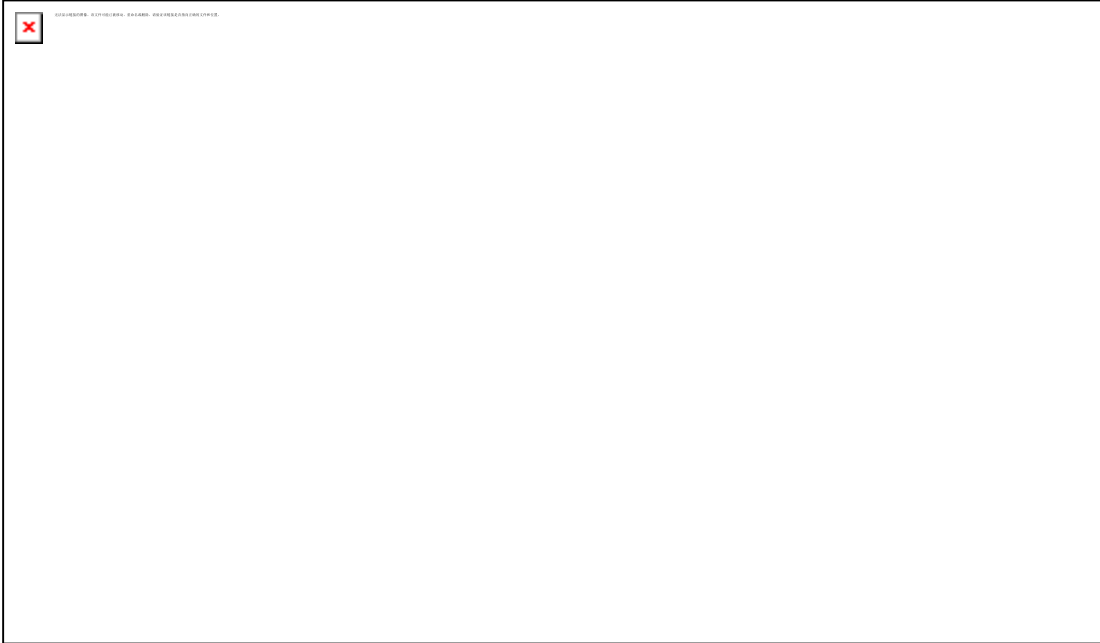
Chapter 2 Software Design

2.1 Rejected Requirements

N/A

2.2 File Structure





2.3 Macros

2.3.1 Macros in Uart_Types.h

- **#define UART_E_UNINIT** ((uint8)0x01U)
Driver not initialized.
- **#define UART_E_INVALID_CHANNEL** ((uint8)0x02U)
Invalid channel given as parameter.
- **#define UART_E_INVALID_POINTER** ((uint8)0x03U)
Invalid channel given as parameter.
- **#define UART_E_ALREADY_INITIALIZED** ((uint8)0x04U)
Driver already initialized.
- **#define UART_E_PARAM_CONFIG** ((uint8)0x05U)
Invalid coreid.
- **#define UART_E_CHANNEL_BUSY** ((uint8)0x06U)
Channel is busy, therefore the service can not be executed.
- **#define UART_E_INVALID_PARAMETER** ((uint8)0x07U)
Parameter has a wrong value.
- **#define UART_E_TIMEOUT** ((uint8)0x08U)
Timeout occurred.

- **#define UART_E_INIT_FAILED** ((uint8)0x09U)
Invalid pointer for init function parameter.
- **#define UART_E_DEINIT_FAILED** ((uint8)0x10U)
De-Init function for specific HW channel finish unsuccessfully.
- **#define UART_INIT_ID** ((uint8)0x00U)
API service ID for UART_Init() function
- **#define UART_DEINIT_ID** ((uint8)0x01U)
API service ID for UART_DeInit() function.
- **#define UART_SETBAUDRATE_ID** ((uint8)0x02U)
API service ID for UART_SetBaudrate() function.
- **#define UART_GETBAUDRATE_ID** ((uint8)0x03U)
API service ID for UART_GetBaudrate() function.
- **#define UART_SYNCSEND_ID** ((uint8)0x04U)
API service ID for UART_SyncSend() function.
- **#define UART_ASYNCSEND_ID** ((uint8)0x05U)
API service ID for UART_AsyncReceive() function.
- **#define UART_SYNCRECEIVE_ID** ((uint8)0x06U)
API service ID for UART_SyncReceive() function.
- **#define UART_ASYNCRECEIVE_ID** ((uint8)0x07U)
API service ID for UART_AsyncSend() function.
- **#define UART_ABORTTRANSFER_ID** ((uint8)0x08U)
API service ID for **Uart_AbortTransfer()** function.
- **#define UART_GETSTATUS_ID** ((uint8)0x09U)
API service ID for UART_GetStatus() function.
- **#define UART_VERSIONINFO_ID** ((uint8)0x0AU)

API service ID for **Uart_GetVersionInfo()** function.

- #define **UART_SETTRANSBUFFER_ID** ((uint8)0x0BU)
API service ID for UART_SetTransBuffer() function.

2.3.2 Macros in Uart_Version.h

- #define **UART_AR_RELEASE_MAJOR_VERSION** 4
- #define **UART_AR_RELEASE_MINOR_VERSION** 6
- #define **UART_AR_RELEASE_REVISION_VERSION** 0
- #define **UART_SW_MAJOR_VERSION** 1
- #define **UART_SW_MINOR_VERSION** 0
- #define **UART_SW_PATCH_VERSION** 3
- #define **UART_VENDOR_ID** 174
- #define **UART_MODULE_ID** 255

2.4 Enums

2.4.1 Enums in Uart_GeneralTypes.h

2.4.1.1 Uart_BitCountPerCharType

Enumeration	Uart_BitCountPerCharType	
Description	UART data bit length mode.	
Values	Value	Description
	UART_7_BITS_PER_CHAR	7-bit data characters
	UART_8_BITS_PER_CHAR	8-bit data characters
	UART_9_BITS_PER_CHAR	9-bit data characters
	UART_10_BITS_PER_CHAR	10-bit data characters

2.4.1.2 Uart_StopBitCountType

Enumeration	Uart_StopBitCountType	
Description	UART stop bits number.	
Values	Value	Description
	UART_ONE_STOP_BIT	UART_STOPBIT_NUM_1
	UART_TWO_STOP_BIT	UART_STOPBIT_NUM_2

2.4.1.3 Uart_ParityModeType

Enumeration	Uart_ParityModeType	
Description	UART parity check type.	
Values	Value	Description
	UART_DISABLED_PARITY	UART_PARITY_DISABLED
	UART_EVEN_PARITY	UART_PARITY_EVEN
	UART_ODD_PARITY	UART_PARITY_ODD

2.4.1.4 Uart_TransferType

Enumeration	Uart_TransferType
-------------	-------------------

Description	Type of UART transfer (based on interrupts or DMA).	
Values	Value	Description
	UART_USING_DMA	The driver will use DMA to perform UART transfer.
	UART_USING_INTERRUPTS	The driver will use interrupts to perform UART transfer.

2.4.1.5 Uart_StatusType

Enumeration	Uart_StatusType	
Description	Driver status type.	
Values	Value	Description
	UART_STATUS_SUCCESS	Uart operation success status.
	UART_STATUS_ERROR	Uart operation failure status.
	UART_STATUS_BUSY	Uart operation busy status.
	UART_STATUS_TIMEOUT	Uart operation timeout status.
	UART_STATUS_TX_UNDERRUN	TX underrun error.
	UART_STATUS_RX_OVERRUN	RX overrun error.
	UART_STATUS_ABORTED	A transfer was aborted
	UART_STATUS_FRAMING_ERROR	Framing error.
	UART_STATUS_PARITY_ERROR	Parity error.
	UART_STATUS_NOISE_ERROR	Noise error.
	UART_STATUS_DMA_ERROR	DMA error.

2.4.1.6 Uart_EventType

Enumeration	Uart_EventType	
Description	Define the enum of the Events which can trigger UART callback.	
Values	Value	Description
	UART_EVENT_RX_FULL	Rx buffer is full.
	UART_EVENT_TX_EMPTY	Tx buffer is empty.
	UART_EVENT_END_TRANSFER	The current transfer is ending.
	UART_EVENT_ERROR	An error occurred during transfer.

2.4.1.7 Uart_BaudrateType

Enumeration	Uart_BaudrateType	
Description	Baudrate values supported by Uart driver.	
Values	Value	Description
	UART_BAUDRATE_1200	The baud rate
	UART_BAUDRATE_2400	The baud rate
	UART_BAUDRATE_4800	The baud rate
	UART_BAUDRATE_7200	The baud rate
	UART_BAUDRATE_9600	The baud rate
	UART_BAUDRATE_14400	The baud rate
	UART_BAUDRATE_19200	The baud rate
	UART_BAUDRATE_28800	The baud rate
	UART_BAUDRATE_38400	The baud rate
	UART_BAUDRATE_57600	The baud rate
	UART_BAUDRATE_115200	The baud rate
	UART_BAUDRATE_230400	The baud rate

	UART_BAUDRATE_460800	The baud rate
	UART_BAUDRATE_921600	The baud rate
	UART_BAUDRATE_1843200	The baud rate

2.4.2 Enums in Uart_Types.h

2.4.2.1 Uart_DirectionType

Enumeration	Uart_DirectionType	
Description	The type operation of an Uart channel.	
Values	Value	Description
	UART_SEND	The sending operation.
	UART_RECEIVE	The receiving operation.

2.5 Typedefs

2.5.1 Typedefs in Uart_Types.h.

- typedef void(* Uart_CallbackType) (const uint8 HwInstance, const Uart_EventType Event)
Callback for all peripherals which support UART features.

2.6 Structures

2.6.1 Uart_StateStructureType

Structure	Uart_StateStructureType
Description	Runtime of the UART driver.
Diagram	N/A
Data Fields	• uint32 BaudRate Variable that indicates if structure belongs to an instance already initialized. • const uint8 * TxBuff The buffer of data being sent. • uint8 * RxBuff The buffer of received data. • volatile uint32 TxSize The remaining number of bytes to be transmitted. • volatile uint32 RxSize The remaining number of bytes to be received. • volatile boolean IsTxBusy True if there is an active transmit. • volatile boolean IsRxBusy True if there is an active receive. • volatile Uart_StatusType TransmitStatus Status of last driver transmit operation. • volatile Uart_StatusType ReceiveStatus Status of last driver receive operation.

2.6.2 Uart_UserConfigType

Structure	Uart_UserConfigType
Description	UART configuration structure.
Diagram	N/A
Data Fields	• uint32 BaudRate Baudrate value. • uint32 BaudRateDivisor Baud clock divisor. • uint8 BaudOverSamplingRatio Over sampling ratio. • Uart_ParityModeType ParityMode Parity mode, disabled (default), even, odd.

	• <code>Uart_StopBitCountType</code> <code>StopBitsCount</code> Number of stop bits, 1 stop bit (default) or 2 stop bits. • <code>Uart_BitCountPerCharType</code> <code>BitCountPerChar</code> Number of bits in a character (8-default, 9 or 10); for 9/10 bits chars, users must provide appropriate buffers to the send/receive functions (bits 8/9 in subsequent bytes); for DMA transmission only 8-bit char is supported. • <code>Uart_TransferType</code> <code>TransferType</code> • <code>Uart_IdleCharNumType</code> <code>IdleCharNum</code> Number of idle characters to detect • <code>Uart_CallbackType</code> <code>Callback</code> Callback to invoke for handle uart event. • <code>Dma_InstanceType</code> <code>eRxDmaInstance</code> • <code>uint32</code> <code>RxDMAChannel</code> DMA channel number for DMA-based rx. If DMA mode is not used this field will be ignored. • <code>Dma_InstanceType</code> <code>eTxDmaInstance</code> • <code>uint32</code> <code>TxDMAChannel</code> DMA channel number for DMA-based tx. If DMA mode is not used this field will be ignored. • <code>Uart_StateStructureType</code> * <code>StateStruct</code>
--	---

2.6.3 Uart_HwConfigType

Structure	<code>Uart_HwConfigType</code>
Description	Structure which brings together all the hardware specific information regarding a channel
Diagram	N/A
Data Fields	• <code>uint32</code> <code>UartHwChannel</code> Uart Hardware Channel. • <code>const Uart_UserConfigType</code> * <code>UserConfig</code> Pointer to the UART_TEXT_SECTION void <code>Uart_configuration</code> structure.

2.6.4 Uart_ChannelConfigType

Structure	<code>Uart_ChannelConfigType</code>
Description	Uart channel configuration type structure.
Diagram	N/A
Data Fields	• <code>uint8</code> <code>UartChannelId</code> Uart channel configured

	- uint32 ChannelCoreId Core id on which the Uart channel has been assigned - uint32 ChannelClockFrequency The clock frequency configured on the given channel - const Uart_HwConfigType * UartChannelConfig Pointer to a lower level channel configuration
--	--

2.6.5 Uart_ConfigType

Structure	Uart_ConfigType
Description	Uart driver configuration type structure.
Diagram	N/A
Data Fields	- uint32 PartitionCoreId Partition configured of the core - const Uart_ChannelConfigType * Configs [UART_CH_MAX_CONFIG] Hardware channel.

2.7 API Functions

2.7.1 Functions in CDD_Uart.h

2.7.1.1 void Uart_Init (const Uart_ConfigType* Config);

Function	void Uart_Init (const Uart_ConfigType * Config)
Description	Initializes the UART module.

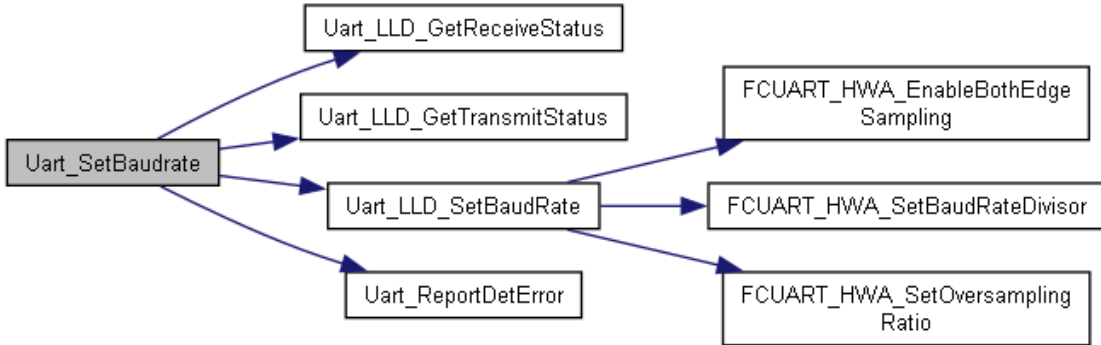
Diagram		
Parameters	Parameter	Description
	Config	Init configuration.
Returns	N/A	

2.7.1.2 void Uart_Deinit(void);

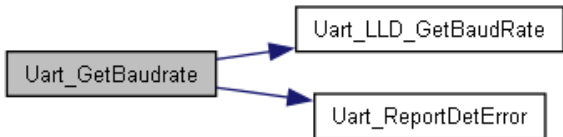
Function	void Uart_Deinit(void);
Description	De-initializes the UART module.

Diagram		
Parameters	Parameter	Description
	N/A	N/A
Returns	N/A	

2.7.1.3 Std_ReturnType Uart_SetBaudrate(uint8 Channel, Uart_BaudrateType Baudrate);

Function	Std_ReturnType Uart_SetBaudrate(uint8 Channel, Uart_BaudrateType Baudrate);	
Description	Configures the baud rate for the serial communication.	
Diagram	 <pre>graph LR; Uart_SetBaudrate[Uart_SetBaudrate] --> Uart_LLD_GetReceiveStatus[Uart_LLD_GetReceiveStatus]; Uart_SetBaudrate --> Uart_LLD_GetTransmitStatus[Uart_LLD_GetTransmitStatus]; Uart_SetBaudrate --> Uart_LLD_SetBaudRate[Uart_LLD_SetBaudRate]; Uart_SetBaudrate --> Uart_ReportDetError[Uart_ReportDetError]; Uart_LLD_SetBaudRate --> FCUART_HWA_EnableBothEdgeSampling[FCUART_HWA_EnableBothEdgeSampling]; Uart_LLD_SetBaudRate --> FCUART_HWA_SetBaudRateDivisor[FCUART_HWA_SetBaudRateDivisor]; Uart_LLD_SetBaudRate --> FCUART_HWA_SetOversamplingRatio[FCUART_HWA_SetOversamplingRatio];</pre>	
Parameters	Parameter	Description
	Channel	UART channel to be addressed.
	Baudrate	Baudrate value to be set.
Returns	Std_ReturnType	

2.7.1.4 Std_ReturnType Uart_GetBaudrate(uint8 Channel, uint32 *Baudrate);

Function	Std_ReturnType Uart_GetBaudrate(uint8 Channel, uint32 *Baudrate);	
Description	Retrieves the baud rate which is currently set for the serial communication.	
Diagram	 <pre>graph LR; Uart_GetBaudrate[Uart_GetBaudrate] --> Uart_LLD_GetBaudRate[Uart_LLD_GetBaudRate]; Uart_GetBaudrate --> Uart_ReportDetError[Uart_ReportDetError];</pre>	
Parameters	Parameter	Description
	Channel	UART channel to be addressed.
	Baudrate	Pointer to a memory location where the baudrate value is returned.
Returns	Std_ReturnType	

2.7.1.5 void Uart_SetTransBuffer(uint8 Channel, uint8 *DataBuffer, uint32 DataSize, Uart_DirectionType TransType);

Function	void Uart_SetTransBuffer(uint8 Channel, uint8 *DataBuffer, uint32 DataSize, Uart_DirectionType TransType);	
Description	Configures a new buffer for continuous transfers.	
Diagram	<pre> graph LR Uart_SetTransBuffer --> Uart_LLD_SetRxBuffer Uart_SetTransBuffer --> Uart_LLD_SetTxBuffer Uart_SetTransBuffer --> Uart_ReportDetError </pre>	
Parameters	Parameter	Description
	Channel	UART channel to be addressed.
	DataBuffer	Buffer to send.
	DataSize	The number of bytes to transmit.
	TransType	Transmission type.
Returns	N/A	

2.7.1.6 Std_ReturnType Uart_SyncSend(uint8 Channel, const uint8 *DataBuffer, uint32 DataSize, uint32 Timeout);

Function	Std_ReturnType Uart_SyncSend(uint8 Channel, const uint8 *DataBuffer, uint32 DataSize, uint32 Timeout);	
Description	Starts a synchronous transfer of bytes.	
Diagram	<pre> graph LR Uart_SyncSend --> Uart_LLD_GetTransmitStatus Uart_SyncSend --> Uart_LLD_SyncSend Uart_SyncSend --> Uart_ReportDetError Uart_LLD_SyncSend --> FCUART_HWA_DisableTransmitCompleteInterrupt Uart_LLD_SyncSend --> FCUART_HWA_DisableTransmitInterrupt Uart_LLD_SyncSend --> FCUART_HWA_GetStatus Uart_LLD_SyncSend --> FCUART_HWA_SetTxTransfer Uart_LLD_SyncSend --> Uart_LLD_PutData Uart_LLD_PutData --> FCUART_HWA_SetData Uart_LLD_PutData --> FCUART_HWA_SetR8T9 Uart_LLD_PutData --> FCUART_HWA_SetR9T8 </pre>	
Parameters	Parameter	Description
	Channel	UART channel to be addressed.
	DataBuffer	The buffer which contains the bytes to be sent The buffer which contains the bytes to be sent.
	DataSize	The Buffer size.
	Timeout	Number of timeout cycles.
Returns	Std_ReturnType	

2.7.1.7 Std_ReturnType Uart_SyncReceive(uint8 Channel, const uint8 *DataBuffer, uint32 DataSize, uint32 Timeout);

Function	Std_ReturnType Uart_SyncReceive(uint8 Channel, uint8 *DataBuffer, uint32 DataSize, uint32 Timeout);	
Description	Starts a synchronous reception of bytes.	
Diagram	<pre> graph LR Uart_SyncReceive --> Uart_LLD_GetReceiveStatus Uart_SyncReceive --> Uart_LLD_SyncReceive Uart_SyncReceive --> Uart_ReportDetError Uart_LLD_SyncReceive --> FCUART_HWA_DisableReceiveInterrupt Uart_LLD_SyncReceive --> FCUART_HWA_GetStatus Uart_LLD_SyncReceive --> FCUART_HWA_SetRxTransfer Uart_LLD_SyncReceive --> FCUART_HWA_WriteClearSTAT Uart_LLD_SyncReceive --> Uart_LLD_GetData Uart_LLD_GetData --> FCUART_HWA_GetR8T9 Uart_LLD_GetData --> FCUART_HWA_GetR9T8 FCUART_HWA_GetStatus --> FCUART_HWA_GetData </pre>	
Parameters	Parameter	Description
	Channel	UART channel to be addressed.
	DataBuffer	The buffer where the bytes will be located.
	DataSize	The Buffer size.
	Timeout	Number of timeout cycles.
Returns	Std_ReturnType	

2.7.1.8 Std_ReturnType Uart_AbortTransfer(uint8 Channel, Uart_DirectionType TransType);

Function	Std_ReturnType Uart_AbortTransfer(uint8 Channel, Uart_DirectionType TransType);	
Description	Aborts an on-going transfer.	
Diagram	<pre> graph LR Uart_AbortTransfer --> Uart_LLD_AbortReceivingData Uart_AbortTransfer --> Uart_LLD_AbortSendingData Uart_LLD_AbortReceivingData --> FCUART_HWA_WriteClearSTAT Uart_LLD_AbortReceivingData --> Uart_LLD_CompleteReceiveDataUsingInt Uart_LLD_AbortReceivingData --> Uart_LLD_CompleteReceiveUsingDma Uart_LLD_AbortReceivingData --> FCUART_HWA_FlushRxBuffer Uart_LLD_AbortReceivingData --> FCUART_HWA_DisableReceiveInterrupt Uart_LLD_AbortReceivingData --> FCUART_HWA_DisableErrorInterrupt Uart_LLD_AbortReceivingData --> FCUART_HWA_GetData Uart_LLD_AbortReceivingData --> FCUART_HWA_SetRxTransfer Uart_LLD_AbortReceivingData --> FCUART_HWA_DisableReceiveDMA Uart_LLD_AbortReceivingData --> FCUART_HWA_GetStatus Uart_LLD_AbortSendingData --> Uart_LLD_CompleteSendUsingDma Uart_LLD_AbortSendingData --> FCUART_HWA_FlushTxBuffer Uart_LLD_AbortSendingData --> Uart_LLD_CompleteSendDataUsingInt Uart_LLD_AbortSendingData --> FCUART_HWA_DisableTransmitDMA Uart_LLD_AbortSendingData --> FCUART_HWA_SetTxTransfer Uart_LLD_AbortSendingData --> FCUART_HWA_DisableTransmitCompleteInterrupt Uart_LLD_AbortSendingData --> FCUART_HWA_DisableTransmitInterrupt Uart_LLD_CompleteReceiveDataUsingInt --> FCUART_HWA_GetData Uart_LLD_CompleteReceiveUsingDma --> FCUART_HWA_SetRxTransfer Uart_LLD_CompleteSendUsingDma --> FCUART_HWA_SetTxTransfer Uart_LLD_CompleteSendDataUsingInt --> FCUART_HWA_DisableTransmitCompleteInterrupt </pre>	
Parameters	Parameter	Description
	Channel	UART channel to be addressed.

	TransType	Type of the transfer to be aborted. It can be either UART_SEND or UART_RECEIVE.
Returns	Std_ReturnType	

2.7.1.9 Std_ReturnType Uart_AsyncSend(uint8 Channel, const uint8 *DataBuffer, uint32 DataSize);

Function	Std_ReturnType Uart_AsyncSend(uint8 Channel, const uint8 *DataBuffer, uint32 DataSize);	
Description	Starts an asynchronous transfer(send) of bytes.	
Diagram	<pre> graph LR Uart_AsyncSend --> Uart_LLD_GetTransmitStatus Uart_AsyncSend --> Uart_ReportDetError Uart_LLD_GetTransmitStatus --> Uart_LLD_AsyncSend Uart_LLD_AsyncSend --> FCUART_HWA_EnableTransmitInterrupt[FCUART_HWA_EnableTransmit Interrupt] Uart_LLD_AsyncSend --> FCUART_HWA_SetTxTransfer Uart_LLD_AsyncSend --> Uart_LLD_StartSendDataUsingDma Uart_LLD_StartSendDataUsingDma --> FCUART_HWA_EnableTransmitDMA </pre>	
Parameters	Parameter	Description
	Channel	Uart channel to be addressed.
	DataBuffer	The buffer where the data to be sent is located.
	DataSize	The Buffer size.
Returns	Std_ReturnType	

2.7.1.10 Std_ReturnType Uart_AsyncReceive (uint8 Channel, const uint8 *DataBuffer, uint32 DataSize);

Function	Std_ReturnType Uart_AsyncSend(uint8 Channel, const uint8 *DataBuffer, uint32 DataSize);	
Description	Starts an asynchronous transfer(receive) of bytes.	
Diagram	<pre> graph LR Uart_AsyncReceive --> Uart_LLD_GetReceiveStatus Uart_AsyncReceive --> Uart_ReportDetError Uart_LLD_GetReceiveStatus --> Uart_LLD_AsyncReceive Uart_LLD_AsyncReceive --> FCUART_HWA_EnableErrorInterrupt Uart_LLD_AsyncReceive --> FCUART_HWA_FlushRxBuffer Uart_LLD_AsyncReceive --> FCUART_HWA_EnableReceiveDMA Uart_LLD_AsyncReceive --> FCUART_HWA_SetRxTransfer Uart_LLD_AsyncReceive --> FCUART_HWA_WriteClearSTAT Uart_LLD_AsyncReceive --> Uart_LLD_StartReceiveDataUsingDma[Uart_LLD_StartReceiveData UsingDma] Uart_LLD_StartReceiveDataUsingDma --> FCUART_HWA_EnableReceiveInterrupt[FCUART_HWA_EnableReceive Interrupt] </pre>	
Parameters	Parameter	Description
	Channel	Uart channel to be addressed.
	DataBuffer	The buffer where the data to be received will located.
	DataSize	The Buffer size.
Returns	Std_ReturnType	

2.7.1.11 Uart_StatusType Uart_GetStatus(uint Channel, uint32 *RemainingBytesPointer, Uart_DirectionType TransType);

Function	Uart_StatusType Uart_GetStatus(uint Channel, uint32 *RemainingBytesPointer, Uart_DirectionType TransType);	
Description	Returns the status of the previous transfer.	

Diagram	<pre>graph LR; Uart_GetStatus[Uart_GetStatus] --> Uart_LLD_GetReceiveStatus[Uart_LLD_GetReceiveStatus]; Uart_GetStatus --> Uart_LLD_GetTransmitStatus[Uart_LLD_GetTransmitStatus]; Uart_GetStatus --> Uart_ReportDetError[Uart_ReportDetError];</pre>	
Parameters	Parameter	Description
	Channel	Uart channel to be addressed.
	RemainingBytesPointer	A pointer where the number of remaining bytes will be written.
	TransType	The type of transfer in discussion (UART_SEND or UART_RECEIVE).
Returns	Uart_StatusType	

2.7.1.12 void Uart_GetVersionInfo(Std_VersionInfoType *VersionInfo);

Function	void Uart_GetVersionInfo(Std_VersionInfoType *VersionInfo);	
Description	Returns the version information of this module.	
Diagram	<pre>graph LR; A[Uart_GetVersionInfo] --> B[Uart_ReportDetError]</pre>	
Parameters	Parameter	Description
	VersionInfo	Pointer for storing the version information of this module.
Returns	N/A	

2.8 Hardware Functions

2.8.1 Functions in LLD_Uart.h

2.8.1.1 void Uart_LLD_Init (const uint8 HwUnit, const Uart_UserConfigType *UserConfig)

Function	void Uart_LLD_Init (const uint8 HwUnit, const Uart_UserConfigType *UserConfig)	
Description	Initializes an UART operation HwUnit.	
Parameters	Parameter	Description
	HwUnit.	UART HwUnit number.
	UserConfig	User configuration structure of type Uart_UserConfigType
Returns	N/A	
Referenced By	Uart_Init()	

2.8.1.2 Uart_StatusType Uart_LLD_Deinit (const uint8 HwUnit)

Function	Uart_StatusType Uart_LLD_Deinit (const uint8 HwUnit)	
Description	Shuts down the UART by disabling interrupts and transmitter/receiver.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
Returns	Uart_StatusType	
Referenced By	Uart_Deinit()	

2.8.1.3 Uart_StatusType Uart_LLD_SyncSend (const uint8 HwUnit, const uint8 *TxDataBuff, const uint32 TxDataSize, const uint32 Timeout)

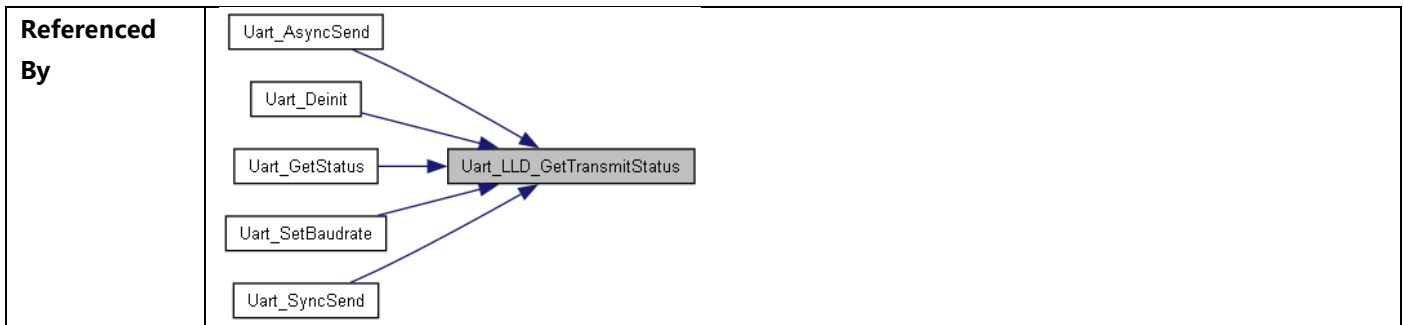
Function	Uart_StatusType Uart_LLD_SyncSend (const uint8 HwUnit, const uint8 *TxDataBuff, const uint32 TxDataSize, const uint32 Timeout)	
Description	Send out multiple bytes of data using polling method.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
	TxDataBuff	The buffer pointer which saves the data to be sent.
	TxDataSize	Size of data to be sent in unit of byte.
	Timeout	Value in microseconds.
Returns	Uart_StatusType	
Referenced By	Uart_SyncSend()	

2.8.1.4 Uart_StatusType Uart_LLD_AsyncSend (const uint8 HwUnit, const uint8 *TxDataBuff, const uint32 TxDataSize)

Function	Uart_StatusType Uart_LLD_AsyncSend (const uint8 HwUnit, const uint8 *TxDataBuff, const uint32 TxDataSize)	
Description	Sends data out through the UART module using a non-blocking method. This enables an a-sync method for transmitting data. When used with a non-blocking receive, the UART can perform a full duplex operation. Non-blocking means that the function returns immediately. The application has to get the transmit status to know when the transmit is complete. Send out multiple bytes of data using polling method.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
	TxDataBuff	The buffer pointer which saves the data to be sent.
	TxDataSize	Size of data to be sent in unit of byte.
Returns	Uart_StatusType	
Referenced By	Uart_AsyncSend()	

2.8.1.5 Uart_StatusType Uart_LLD_GetTransmitStatus (const uint8 HwUnit, uint32 *BytesRemaining)

Function	Uart_StatusType Uart_LLD_GetTransmitStatus (const uint8 HwUnit, uint32 *BytesRemaining)	
Description	Returns whether the previous transmit is complete.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
	BytesRemaining	Pointer to value that is populated with the number of bytes that have been sent in the active transfer.
Returns	Uart_StatusType	



2.8.1.6 Uart_StatusType Uart_LLD_AbortSendingData (const uint8 HwUnit)

Function	Uart_StatusType Uart_LLD_AbortSendingData (const uint8 HwUnit)	
Description	Terminates a non-blocking transmission early.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number
Returns	Uart_StatusType	
Referenced By	Uart_AbortTransfer()	

2.8.1.7 Uart_StatusType Uart_LLD_SyncReceive (const uint8 HwUnit, uint8 *RxDataBuff, const uint32 RxDataSize, const uint32 Timeout)

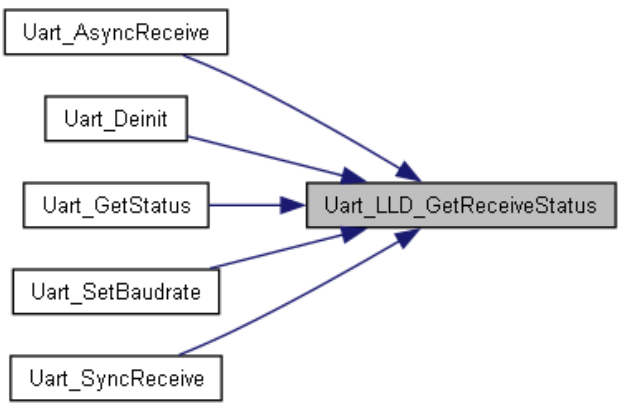
Function	Uart_StatusType Uart_LLD_SyncReceive (const uint8 HwUnit, uint8 *RxDataBuff, const uint32 RxDataSize, const uint32 Timeout)	
Description	Receive multiple bytes of data using polling method.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
	RxDataBuff	The buffer pointer which saves the data to be received.
	RxDataSize	Size of data need to be received in unit of byte.
	Timeout	The number of timeout cycles per frame of data reception.
Returns	Uart_StatusType	
Referenced By	Uart_SyncReceive()	

2.8.1.8 Uart_StatusType Uart_LLD_AsyncReceive(const uint8 HwUnit, uint8 * RxDataBuff, const uint32 RxDataSize);

Function	Uart_StatusType Uart_LLD_AsyncReceive(const uint8 HwUnit, uint8 * RxDataBuff, const uint32 RxDataSize);	
Description	Gets data from the UART module by using a non-blocking method. This enables an a-sync method for receiving data. When used with a non-blocking transmission, the UART can perform a full duplex operation. Non-blocking means that the function returns immediately. The application has to get the receive status to know when the receive is complete.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
	RxDataBuff	The buffer pointer which saves the data to be received.
	RxDataSize	Size of data need to be received in unit of byte.
Returns	N/A	

Referenced By	Uart_AsyncReceive()
----------------------	---------------------

2.8.1.9 Uart_StatusType Uart_LLD_GetReceiveStatus (const uint8 HwUnit, uint32 *BytesRemaining)

Function	Uart_StatusType Uart_LLD_GetReceiveStatus (const uint8 HwUnit, uint32 *BytesRemaining)	
Description	Returns whether the previous receive is complete.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
	BytesRemaining	pointer to value that is filled with the number of bytes that still need to be received in the active transfer.
Returns	Uart_StatusType	
Referenced By	 <pre> graph LR Uart_AsyncReceive --> Uart_LLD_GetReceiveStatus Uart_Deinit --> Uart_LLD_GetReceiveStatus Uart_GetStatus --> Uart_LLD_GetReceiveStatus Uart_SetBaudrate --> Uart_LLD_GetReceiveStatus Uart_SyncReceive --> Uart_LLD_GetReceiveStatus </pre>	

2.8.1.10 Uart_StatusType Uart_LLD_AbortReceivingData (const uint8 HwUnit);

Function	Uart_StatusType Uart_LLD_AbortReceivingData (const uint8 HwUnit);	
Description	Terminates a non-blocking receive early.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number
Returns	N/A	
Referenced By	Uart_AbortTransfer()	

2.8.1.11 Uart_StatusType Uart_LLD_SetBaudRate (const uint8 HwUnit, const Uart_BaudrateType DesiredBaudrate, const uint32 ClockFrequency) ;

Function	Uart_StatusType Uart_LLD_SetBaudRate (const uint8 HwUnit, const Uart_BaudrateType DesiredBaudrate, const uint32 ClockFrequency) ;	
Description	Configures the UART baud rate.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
	DesiredBaudrate	UART desired baud rate.
	ClockFrequency	Clock Frequency of UART HwUnit.
Returns	Uart_StatusType	
Referenced By	Uart_SetBaudrate()	

2.8.1.12 void Uart_LLD_GetBaudRate (const uint8 HwUnit, uint32 * ConfiguredBaudRate);

Function	void Uart_LLD_GetBaudRate (const uint8 HwUnit, uint32 * ConfiguredBaudRate);
-----------------	--

Description	This function returns the UART baud rate.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
	ConfiguredBaudRate	UART configured baud rate.
Returns	N/A	
Referenced By	Uart_GetBaudrate()	

2.8.1.13 void Uart_LLD_SetTxBuffer(const uint8 HwUnit, const uint8 * TxDataBuff, const uint32 TxDataSize);

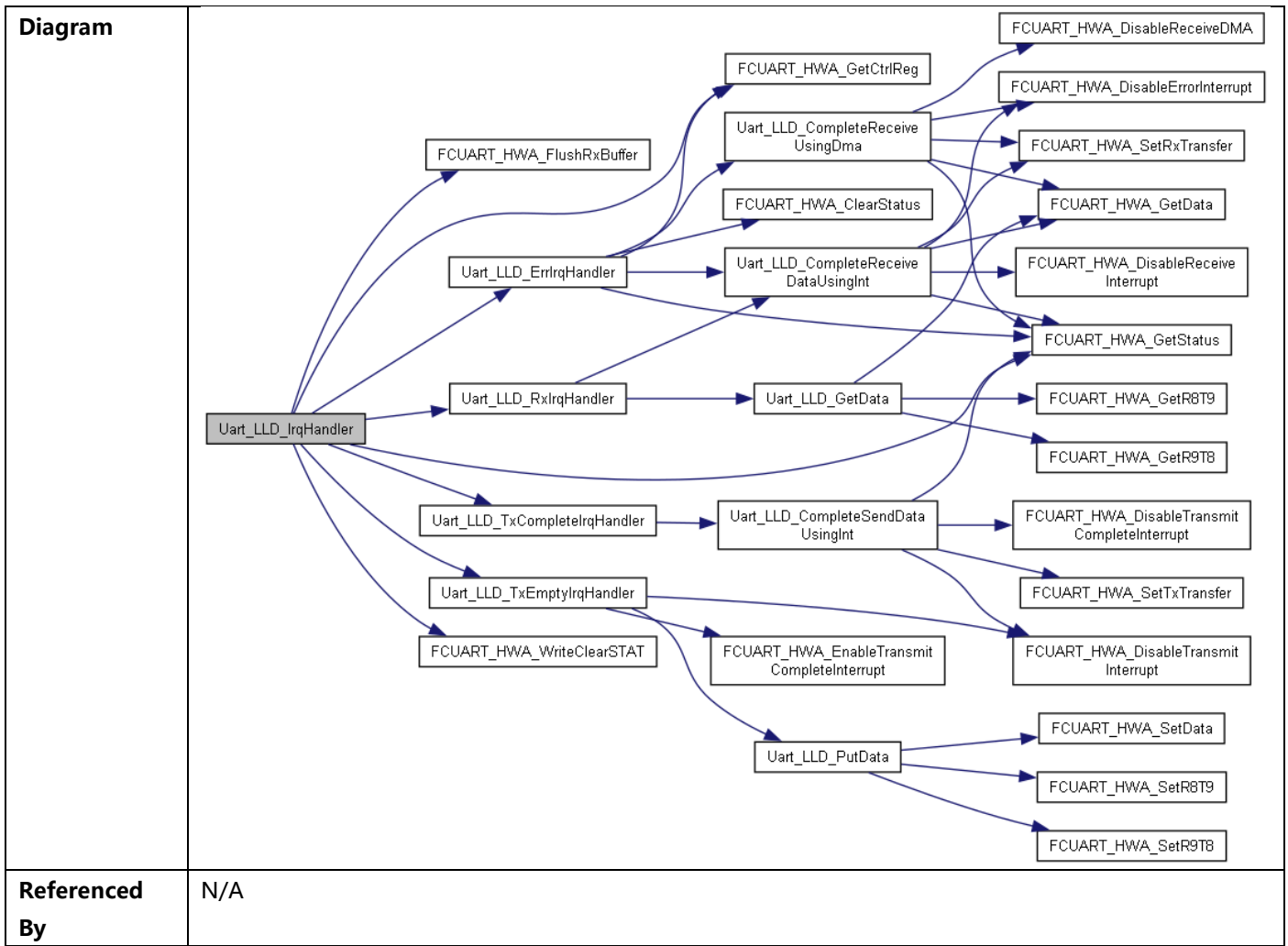
Function	void Uart_LLD_SetTxBuffer(const uint8 HwUnit, const uint8 * TxDataBuff, const uint32 TxDataSize);	
Description	Sets the internal driver reference to the tx buffer.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
	TxDataBuff	source buffer containing 8-bit data chars to send
	TxDataSize	the number of bytes to send
Returns	N/A	
Referenced By	Uart_SetTransBuffer()	

2.8.1.14 void Uart_LLD_SetRxBuffer(const uint8 HwUnit, uint8 * RxDataBuff, const uint32 RxDataSize);

Function	void Uart_LLD_SetRxBuffer(const uint8 HwUnit, uint8 * RxDataBuff, const uint32 RxDataSize);	
Description	Sets the internal driver reference to the rx buffer.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
	RxDataBuff	destination buffer containing 8-bit data chars to receive
	RxDataSize	the number of bytes to receive
Returns	N/A	
Referenced By	Uart_SetTransBuffer()	

2.8.1.15 void Uart_LLD_IrqHandler (const uint8 HwUnit);

Function	void Uart_LLD_IrqHandler (const uint8 HwUnit);	
Description	Interrupt handler for FCUART.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
Returns	N/A	



2.8.1.16 void Uart_LLD_CompleteSendUsingDma(uint8 HwUnit);

Function	void Uart_LLD_CompleteSendUsingDma (uint8 HwUnit);	
Description	Complete dma sending data.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
Returns	N/A	
Referenced By		

2.8.1.17 void Uart_LLD_CompleteReceiveUsingDma(uint8 HwUnit);

Function	void Uart_LLD_CompleteReceiveUsingDma(uint8 HwUnit);	
Description	Finish up a receive by completing the process of receiving data and disabling the DMA requests.	
Parameters	Parameter	Description
	HwUnit	UART HwUnit number.
Returns	N/A	
Referenced By		

2.9 Peripheral Functions

2.9.1 Functions in Uart_RegOps.h

2.9.1.1 LOCAL_INLINE uint32 FCUART_HWA_GetStatus(FCUART_Type *pUart, FCUART_StatType eStatusType)

Function	LOCAL_INLINE uint32 FCUART_HWA_GetStatus(FCUART_Type *pUart, FCUART_StatType eStatusType)	
Description	Get Stat Flag.	
Parameters	Parameter	Description
	pUart	UART instance value.
	eStatusType	Status type.
Returns	FCUART_STAT status flag	
Referenced By	N/A	

2.9.1.2 LOCAL_INLINE void FCUART_HWA_ClearStatus(FCUART_Type *pUart, FCUART_StatType eStatusType)

Function	LOCAL_INLINE void FCUART_HWA_ClearStatus(FCUART_Type *pUart, FCUART_StatType eStatusType)	
Description	Clear Stat Flag.	
Parameters	Parameter	Description
	pUart	UART instance value.
	eStatusType	Status type.
Returns	N/A	
Referenced By	N/A	

2.9.1.3 LOCAL_INLINE void FCUART_HWA_EnableReceiveInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableReceiveInterrupt(FCUART_Type *pUart)	
Description	Enable Receive Interrupt.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.4 LOCAL_INLINE void FCUART_HWA_DisableReceiveInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableReceiveInterrupt(FCUART_Type *pUart)	
Description	Disable Receive Interrupt.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.5 LOCAL_INLINE void FCUART_HWA_EnableErrorInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableErrorInterrupt(FCUART_Type *pUart)	
Description	Enable Error Interrupt(Parity Error Exclude).	
Parameters	Parameter	Description
	pUart	UART instance value.

Returns	N/A
Referenced By	N/A

2.9.1.6 LOCAL_INLINE void FCUART_HWA_DisableErrorInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableErrorInterrupt(FCUART_Type *pUart)	
Description	Disable Error Interrupt(Parity Error Exclude).	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.7 LOCAL_INLINE uint32 FCUART_HWA_GetCtrlReg(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE uint32 FCUART_HWA_GetCtrlReg(FCUART_Type *pUart, uint32 u32Value)	
Description	Get FCUART Ctrl register specified value.	
Parameters	Parameter	Description
	pUart	UART instance value.
	u32Value	Ctrl register specified mask.
Returns	N/A	
Referenced By	N/A	

2.9.1.8 LOCAL_INLINE void FCUART_HWA_EnableTransmitInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableTransmitInterrupt(FCUART_Type *pUart)	
Description	Enable Transmit Interrupt.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.9 LOCAL_INLINE void FCUART_HWA_DisableTransmitInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableTransmitInterrupt(FCUART_Type *pUart)	
Description	Disable Transmit Interrupt.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.10 LOCAL_INLINE void FCUART_HWA_EnableTransmitCompleteInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableTransmitCompleteInterrupt(FCUART_Type *pUart)	
Description	Enable Transmit Complete Interrupt.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.11 LOCAL_INLINE void FCUART_HWA_DisableTransmitCompleteInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableTransmitCompleteInterrupt(FCUART_Type *pUart)	
Description	Disable Transmit Complete Interrupt.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.12 LOCAL_INLINE void FCUART_HWA_SetBaud(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_SetBaud(FCUART_Type *pUart, uint32 u32Value)	
Description	Set FCUART Baud register.	
Parameters	Parameter	Description
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.13 LOCAL_INLINE void FCUART_HWA_SetOversamplingRatio (FCUART_Type *pUart, uint32 OverSamplingRatio)

Function	LOCAL_INLINE void FCUART_HWA_SetOversamplingRatio (FCUART_Type *pUart, uint32 OverSamplingRatio)	
Description	Sets the UART baud rate oversampling ratio.	
Parameters	Parameter	Description
	pUart	UART instance value.
	OverSamplingRatio	The oversampling ratio "OSR"
Returns	N/A	
Referenced By	N/A	

2.9.1.14 LOCAL_INLINE void FCUART_HWA_SetBaudRateDivisor(FCUART_Type *pUart, uint32 baudRateDivisor)

Function	LOCAL_INLINE void FCUART_HWA_SetBaudRateDivisor(FCUART_Type *pUart, uint32 baudRateDivisor)	
Description	Sets the UART baud rate oversampling ratio.	
Parameters	Parameter	Description
	pUart	UART instance value.
	baudRateDivisor	The baud rate modulo division "SBR"
Returns	N/A	
Referenced By	N/A	

2.9.1.15 LOCAL_INLINE void FCUART_HWA_EnableBothEdgeSampling(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableBothEdgeSampling(FCUART_Type *pUart)	
Description	Enable Both Edge Sampling.	

Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.16 LOCAL_INLINE void FCUART_HWA_SetBitCountPerChar(FCUART_Type *pUart, Uart_BitCountPerCharType BitCountPerChar, boolean Parity)

Function	LOCAL_INLINE void FCUART_HWA_SetBitCountPerChar(FCUART_Type *pUart, Uart_BitCountPerCharType BitCountPerChar, boolean Parity)	
Description	Enable Both Edge Sampling.	
Parameters	Parameter	Description
	pUart	UART instance value.
	BitCountPerChar	Number of bits per char.
	Parity	Specifies whether parity bit is enabled.
Returns	N/A	
Referenced By	N/A	

2.9.1.17 LOCAL_INLINE void FCUART_HWA_WriteClearSTAT(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_WriteClearSTAT(FCUART_Type *pUart, uint32 u32Value)	
Description	Write 1 Clear FCUART STAT register.	
Parameters	Parameter	Description
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.18 LOCAL_INLINE void FCUART_HWA_SetParity(FCUART_Type *pUart, Uart_ParityModeType eParityType)

Function	LOCAL_INLINE void FCUART_HWA_SetParity(FCUART_Type *pUart, Uart_ParityModeType eParityType)	
Description	Set Parity.	
Parameters	Parameter	Description
	pUart	UART instance value.
	eParityType	Parity mode (enabled, disable, odd, even)
Returns	N/A	
Referenced By	N/A	

2.9.1.19 LOCAL_INLINE void FCUART_HWA_SetStopBit(FCUART_Type *pUart, Uart_StopBitCountType eStopBit)

Function	LOCAL_INLINE void FCUART_HWA_SetStopBit(FCUART_Type *pUart, Uart_StopBitCountType eStopBit)	
Description	Set Parity.	
Parameters	Parameter	Description
	pUart	UART instance value.
	eStopBit	stop bits number 1 or 2 bits

Returns	N/A
Referenced By	N/A

2.9.1.20 LOCAL_INLINE void FCUART_HWA_EnableTransmitDMA(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableTransmitDMA(FCUART_Type *pUart)	
Description	Enable Transmit DMA	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.21 LOCAL_INLINE void FCUART_HWA_DisableTransmitDMA (FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableTransmitDMA(FCUART_Type *pUart)	
Description	Disable Transmit DMA	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.22 LOCAL_INLINE void FCUART_HWA_EnableReceiveDMA(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableReceiveDMA(FCUART_Type *pUart)	
Description	Enable Receive DMA.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.23 LOCAL_INLINE void FCUART_HWA_DisableReceiveDMA(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableReceiveDMA(FCUART_Type *pUart)	
Description	Disable Receive DMA	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.24 LOCAL_INLINE void FCUART_HWA_FlushTxBuffer(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_FlushTxBuffer(FCUART_Type *pUart)	
Description	Clear FCUART Tx Fifo register	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.25 LOCAL_INLINE void FCUART_HWA_FlushRxBuffer(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_FlushRxBuffer(FCUART_Type *pUart)	
Description	Clear FCUART Rx Fifo register	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.26 LOCAL_INLINE void FCUART_HWA_SetData(FCUART_Type *pUart, uint32 u32Data)

Function	LOCAL_INLINE void FCUART_HWA_SetData(FCUART_Type *pUart, uint32 u32Data)	
Description	Set the data value.	
Parameters	Parameter	Description
	pUart	UART instance value.
	u32Data	Set data.
Returns	N/A	
Referenced By	N/A	

2.9.1.27 LOCAL_INLINE uint8 FCUART_HWA_GetData(FCUART_Type *pUart)

Function	LOCAL_INLINE uint8 FCUART_HWA_GetData(FCUART_Type *pUart)	
Description	Get the data value.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	The data value.	
Referenced By	N/A	

2.9.1.28 LOCAL_INLINE void FCUART_HWA_SetR8T9(FCUART_Type *pUart, uint8 u8Data)

Function	LOCAL_INLINE void FCUART_HWA_SetR8T9(FCUART_Type *pUart, uint8 u8Data)	
Description	Set R8T9 bit.	
Parameters	Parameter	Description
	pUart	UART instance value.
	u8Data	The R8T9 data
Returns	N/A	
Referenced By	N/A	

2.9.1.29 LOCAL_INLINE uint8 FCUART_HWA_GetR8T9(FCUART_Type *pUart)

Function	LOCAL_INLINE uint8 FCUART_HWA_GetR8T9(FCUART_Type *pUart)	
Description	Get R8T9 bit.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	The data value.	
Referenced By	N/A	

2.9.1.30 LOCAL_INLINE void FCUART_HWA_SetR9T8(FCUART_Type *pUart, uint8 u8Data)

Function	LOCAL_INLINE void FCUART_HWA_SetR9T8(FCUART_Type *pUart, uint8 u8Data)	
Description	Set R8T9 bit.	
Parameters	Parameter	Description
	pUart	UART instance value.
	u8Data	The R9T8 data
Returns	N/A	
Referenced By	N/A	

2.9.1.31 LOCAL_INLINE uint8 FCUART_HWA_GetR9T8(FCUART_Type *pUart)

Function	LOCAL_INLINE uint8 FCUART_HWA_GetR9T8(FCUART_Type *pUart)	
Description	Get R9T8 bit.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	The data value.	
Referenced By	N/A	

2.9.1.32 LOCAL_INLINE void FCUART_HWA_SoftwareReset(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_SoftwareReset(FCUART_Type *pUart)	
Description	Reset the instance by software.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.33 LOCAL_INLINE void FCUART_HWA_SetTxTransfer(FCUART_Type *pUart, boolean bEnable);

Function	LOCAL_INLINE void FCUART_HWA_SetTxTransfer(FCUART_Type *pUart, boolean bEnable);	
Description	Set fcuart TX Transfer enable or disable.	
Parameters	Parameter	Description
	pUart	UART instance value.
	bEnable	Enable cmd, false for disable, true for enable.
Returns	N/A	
Referenced By	N/A	

2.9.1.34 LOCAL_INLINE void FCUART_HWA_SetRxTransfer(FCUART_Type *pUart, boolean bEnable);

Function	LOCAL_INLINE void FCUART_HWA_SetRxTransfer(FCUART_Type *pUart, boolean bEnable);	
Description	Set fcuart RX Transfer enable or disable.	
Parameters	Parameter	Description
	pUart	UART instance value.
	bEnable	Enable cmd, false for disable, true for enable.
Returns	N/A	
Referenced By	N/A	

2.9.1.35 LOCAL_INLINE void FCUART_HWA_EnableLoopMode(FCUART_Type *pUart);

Function	LOCAL_INLINE void FCUART_HWA_EnableLoopMode(FCUART_Type *pUart);	
Description	Enable the FCUART loop mode.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.36 LOCAL_INLINE void FCUART_HWA_EnableHwFlowControl(FCUART_Type *pUart);

Function	LOCAL_INLINE void FCUART_HWA_EnableHwFlowControl(UART_Type *pUart);	
Description	Enable the FCUART hardware flow control.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.37 LOCAL_INLINE void FCUART_UART_HWA_EnableIdleInterrupt(FCUART_Type *pUart);

Function	LOCAL_INLINE void FCUART_UART_HWA_EnableHwFlowControl(UART_Type *pUart);	
Description	Enable the FCUART idle interrupt.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.38 LOCAL_INLINE void FCUART_UART_HWA_SetIdleConfig(FCUART_Type *pUart, Uart_IdleCharNumType eIdleCharNum);

Function	LOCAL_INLINE void FCUART_UART_HWA_SetIdleConfig(FCUART_Type *pUart, Uart_IdleCharNumType eIdleCharNum);	
Description	Set the idle character number.	
Parameters	Parameter	Description
	pUart	UART instance value.
	eIdleCharNum	Idle character number
Returns	N/A	
Referenced By	N/A	

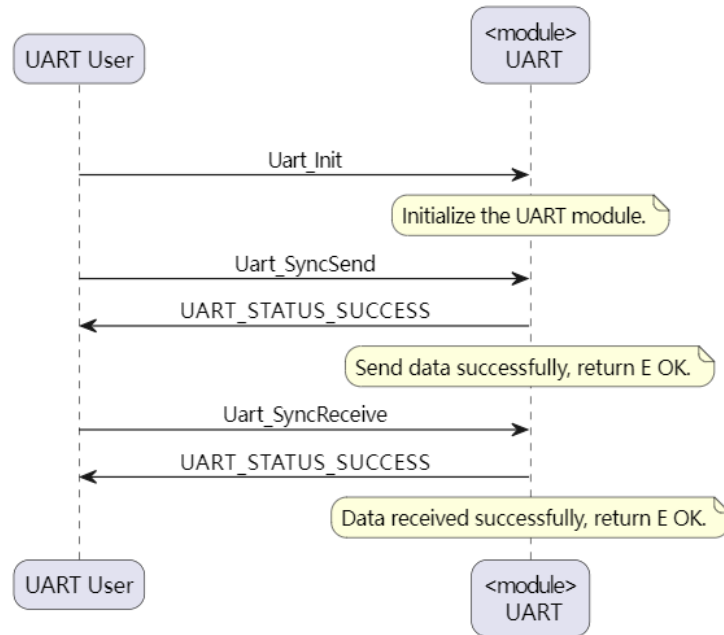
2.9.1.39 LOCAL_INLINE void FCUART_UART_HWA_DisableIdleInterrupt(FCUART_Type *pUart);

Function	LOCAL_INLINE void FCUART_UART_HWA_DisableHwFlowControl(UART_Type *pUart);	
Description	Disable the FCUART idle interrupt.	
Parameters	Parameter	Description
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.10 API Sequence Diagram

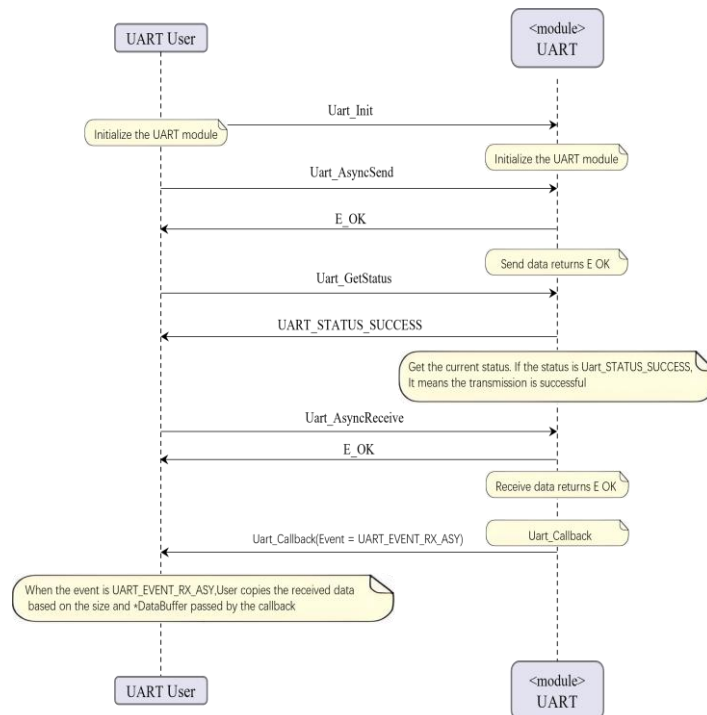
2.10.1 Synchronous transmission

The following figure shows synchronous UART transmission. After initialization, the Uart_SyncSend interface is called to send data, and the Uart_SyncReceive interface is called to receive data.



2.10.2 Asynchronous transmission

The following figure shows the general usage of asynchronous transmission. After initialization, the send or transmit interface is called, and Uart_GetStatus is called to obtain the current status. When the status is UART_STATUS_SUCCESS, it indicates successful reception.



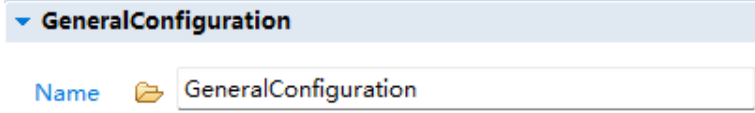
Chapter 3 Tresos Configuration Items

3.1 Containers and Variables

3.1.1 IMPLEMENTATION_CONFIG_VARIANT

Container	IMPLEMENTATION_CONFIG_VARIANT	
Description	N/A	
Screenshot		
Properties	Property	Value
	Type	Variable: enumeration
	Symbolic Name	False
	Default	VariantPostBuild
	Range	VariantPostBuild, VariantPreCompile
Returns	N/A	

3.1.2 GeneralConfiguration

Container	GeneralConfiguration	
Description	This container contains the global configuration parameters of the Non-Autosar Uart driver.	
Screenshot		
Properties	Property	Value
	Type	Container

3.1.2.1 UartDevErrorDetect

Container	UartDevErrorDetect	
Description	Switches the Development Error Detection and Notification ON or OFF.	
Screenshot		
Properties	Property	Value
	Type	Variable:boolean
	Symbolic Name	False
	Default	False

3.1.2.2 DisableUartRuntimeErrorDetect

Container	DisableUartRuntimeErrorDetect	
Description	Switches the Runtime Error Detection and Notification ON or OFF.	
Screenshot		
Properties	Property	Value
	Type	Variable:boolean
	Symbolic Name	False
	Default	False

3.1.2.3 UartMulticoreEnable

Container	UartMulticoreEnable	
Description	When this parameter is enabled, multi-core feature will be used in Uart driver. That means mapping the Uart driver to multiple ECUC partitions to make the module API available in this partition.	
Screenshot	Uart Multicore Support 	
Properties	Property	Value
	Type	Variable:boolean
	Symbolic Name	False
	Default	False

3.1.2.4 UartTimeoutDuration

List	UartTimeoutDuration	
Description	Specifies the maximum number of loops for blocking function until a timeout is raised in short term wait loops. The UartTimeoutDuration is number of wait loop.	
Screenshot	Uart Timeout Duration (0 -> 4294967295) 	
Properties	Property	Value
	Type	Variable:integer Variable:integer
	Default	1000
	Range	0-4294967295

3.1.2.5 UartDmaEnable

Container	UartDmaEnable	
Description	Check this in order to be able to use DMA in the Uart driver.	
Screenshot	Uart DMA Enable 	
Properties	Property	Value
	Type	Variable:boolean
	Symbolic Name	False
	Default	False

3.1.2.6 UartVersionInfoApi

Container	UartVersionInfoApi	
Description	Switches the Uart_GetVersionInfo function ON or OFF.	
Screenshot	Provide Uart VersionInfo Api 	
Properties	Property	Value
	Type	Variable:boolean
	Symbolic Name	False
	Default	False

3.1.2.7 UartCallbackCapability

Container	UartCallbackCapability	
Description	When this parameter is enabled, Callback feature will be used in Uart driver, with the following functions: a) completion of a reception or sending operation. b) reporting communication errors.	
Screenshot	Uart Callback Capability 	

Properties	Property	Value
	Type	Variable: boolean
	Symbolic Name	False
	Default	False

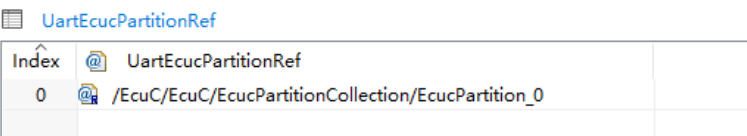
3.1.2.8 UartCallback

Variable	UartCallback	
Description	Callback function of the uart module.	
Screenshot		
Properties	Property	Value
	Type	Variable: String
	Default	NULL_PTR

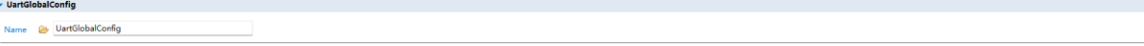
3.1.2.9 Uart ReRecv Enable

Variable	Uart ReRecv Enable	
Description	Uart asynchronously receives a packet of data and resets the receiving status.	
Screenshot		
Properties	Property	Value
	Type	Variable: boolean
	Default	False

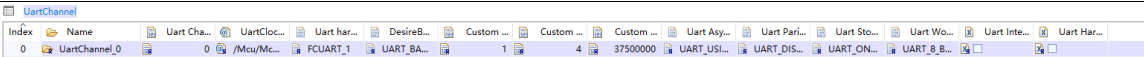
3.1.3 UartEcucPartitionRef

Variable	UartEcucPartitionRef	
Description	This container contains the global configuration parameter of the Uart driver.	
Screenshot		
Properties	Property	Value
	Type	Reference
	destination	ASPathDataOfSchema:/AUTOSAR/EcuC/EcucPartitionCollection/EcucPartition

3.1.4 UartGlobalConfig

Variable	UartGlobalConfig	
Description	This container contains the global configuration parameter of the Uart driver.	
Screenshot		
Properties	Property	Value
	Type	Variable: Container

3.1.4.1.1 UartChannel

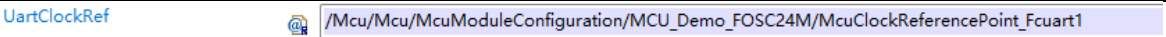
Variable	UartChannel	
Description	This container contains the configuration (parameters) of the Uart Controller(s).	
Screenshot		
Properties	Property	Value
	Type	Variable: Container

	Type	Variable: Container
--	------	---------------------

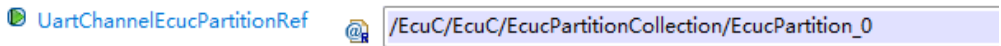
3.1.4.1.2 UartChannelId

Variable	UartChannelId	
Description	Identifies the Uart channel.	
Screenshot		
Properties	Property	Value
	Type	Integer(Range)
	Default	0
	Range	0-18

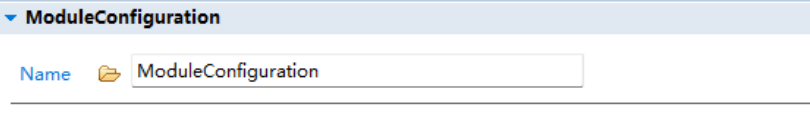
3.1.4.1.3 UartClockRef

Container	UartClockRef	
Description	Reference to the Uart clock source configuration, which is set into the MCU driver configuration. This clock source is used for configure Uart baudrate.	
Screenshot		
Properties	Property	Value
	Type	Reference
	destination	ASPathDataOfSchema:/AUTOSAR/EcuDefs/Mcu/McuModuleConfiguration/McuClockSettingConfig/McuClockReferencePoint

3.1.4.1.4 UartChannelEcucPartitionRef

Container	UartChannelEcucPartitionRef	
Description	Maps one single Uart channel to zero or one ECUC partitions. The ECUC partition referenced is a subset of the ECUC partitions where the Uart driver is mapped to.	
Screenshot		
Properties	Property	Value
	Type	Reference
	destination	ASPathDataOfSchema:/AUTOSAR/EcuDefs/EcuC/EcucPartitionCollection/EcucPartition

3.1.5 ModuleConfiguration

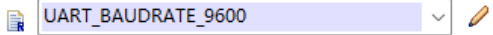
Variable	ModuleConfiguration	
Description	This container contains the configuration (parameters) of the Module Configuration.	
Screenshot		
Properties	Property	Value
	Type	Container

3.1.5.1 UartHwChannel

Variable	UartHwChannel	
Description	Selects the physical Uart Channel.	
Screenshot		

Properties	Property	Value
	Type	Variable: String(Range)
	Default	FCUART_1
	Range	[FCUART_0, FCUART_1, FCUART_2, FCUART_3, FCUART_4, FCUART_5, FCUART_6, FCUART_7, FCUART_8, FCUART_9, FCUART_10, FCUART_11, FCUART_12, FCUART_13, FCUART_14, FCUART_15, FCUART_16, FCUART_17]

3.1.5.2 DesireBaudrate

Variable	DesireBaudrate	
Description	This parameter is user's desire baudrate.	
Screenshot		
Properties	Property	Value
	Type	Variable: String(Range)
	Default	UART_BAUDRATE_9600
	Range	[UART_BAUDRATE_CUSTOM, UART_BAUDRATE_1200, UART_BAUDRATE_2400, UART_BAUDRATE_4800, UART_BAUDRATE_7200, UART_BAUDRATE_9600, UART_BAUDRATE_14400, UART_BAUDRATE_19200, UART_BAUDRATE_28800, UART_BAUDRATE_38400, UART_BAUDRATE_57600, UART_BAUDRATE_115200, UART_BAUDRATE_230400, UART_BAUDRATE_460800, UART_BAUDRATE_921600, UART_BAUDRATE_1843200]

3.1.5.3 CustomBaudrateMantissa

Variable	CustomBaudrateMantissa	
Description	Identifies the custom baudrate value.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Default	1
	Range	1 -> 8191

3.1.5.4 CustomBaudrateDivisor

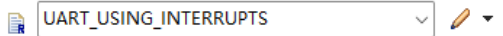
Variable	CustomBaudrateDivisor	
Description	Identifies the custom baudrate value.	
Screenshot		

Properties	Property	Value
	Type	Variable: Integer
	Default	4
	Range	4 -> 32

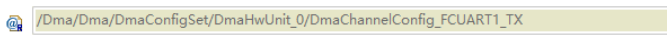
3.1.5.5 CustomBaudrateValue

Variable	CustomBaudrateValue	
Description	Identifies the custom baudrate value.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Default	0
	Range	-9223372036854775808-9223372036854775807

3.1.5.6 UartInteruptDmaMethod

Container	UartInteruptDmaMethod	
Description	Configures the mechanism used by the 'AsyncSend' or 'AsyncReceive' function (interrupts or DMA).	
Screenshot		
Properties	Property	Value
	Type	Variable: String(Range)
	Default	UART_USING_INTERRUPTS
	Range	[UART_USING_INTERRUPTS, UART_USING_DMA]

3.1.5.7 UartDmaTxChannelRef

Container	UartDmaTxChannelRef	
Description	Reference to the DMA TX channel, which is set in the Mcal driver configuration.	
Screenshot		
Properties	Property	Value
	Type	Variable: Reference
	Range	ASPathDataOfSchema:/AUTOSAR/EcucDefs/Dma/DmaConfigSet/DmaHwUnit/DmaChannelConfig

3.1.5.8 UartDmaRxChannelRef

Container	UartDmaRxChannelRef	
Description	Reference to the DMA RX channel, which is set in the Mcal driver configuration.	
Screenshot		
Properties	Property	Value
	Type	Variable: Reference
	Range	ASPathDataOfSchema:/AUTOSAR/EcucDefs/Dma/DmaConfigSet/DmaHwUnit/DmaChannelConfig

3.1.5.9 UartParityType

Variable	UartParityType	
Description	Configures the type of parity to be used for UART bytes.	
Screenshot		

Properties	Property	Value
	Type	Variable: String(Range)
	Default	UART_DISABLED_PARITY
	Range	[UART_DISABLED_PARITY, UART_EVEN_PARITY, UART_ODD_PARITY]

3.1.5.10 UartStopBitNumber

Variable	UartStopBitNumber	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	Property	Value
	Type	Variable: String(Range)
	Default	UART_ONE_STOP_BIT
	Range	[UART_ONE_STOP_BIT, UART_TWO_STOP_BIT]

3.1.5.11 UartWordLength

Variable	UartWordLength	
Description	Configures the word length in UART mode.	
Screenshot		
Properties	Property	Value
	Type	Variable: String(Range)
	Default	UART_8_BITS_PER_CHAR
	Range	[UART_7_BITS_PER_CHAR, UART_8_BITS_PER_CHAR, UART_9_BITS_PER_CHAR, UART_10_BITS_PER_CHAR]

3.1.5.12 UartInternalLoopbackEnable

Variable	UartInternalLoopbackEnable	
Description	This checks enables the internal loopback for the current channel.	
Screenshot		
Properties	Property	Value
	Type	Variable: Boolean
	Default	false
	Symbolic Name	false

3.1.5.13 UartHardwareFlowControlEnable

Variable	UartHardwareFlowControlEnable	
Description	This checks enables the hardware flow control (CTS/RTS) for the current channel	
Screenshot		
Properties	Property	Value
	Type	Variable: Boolean
	Default	false
	Symbolic Name	false

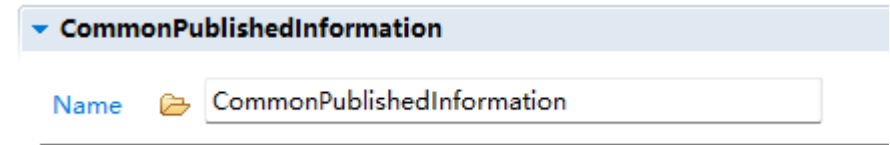
3.1.5.14 UartTimeoutEnable

Variable	UartTimeoutEnable	
Description	This checks enables the Uart idle Interrupt for the current channel.	
Screenshot		
Properties	Property	Value
	Type	Variable: Boolean
	Default	false
	Symbolic Name	false

3.1.5.15 UartIdleNumber

Variable	UartIdleNumber	
Description	Configures the number of idle characters that must be received before the IDLE flag is set.	
Screenshot		
Properties	Property	Value
	Type	Variable: String(Range)
	Default	UART_IDLE_CHARACTER_1
	Range	[UART_IDLE_CHARACTER_1, UART_IDLE_CHARACTER_2, UART_IDLE_CHARACTER_4, UART_IDLE_CHARACTER_8, UART_IDLE_CHARACTER_16, UART_IDLE_CHARACTER_32, UART_IDLE_CHARACTER_64, UART_IDLE_CHARACTER_128]

3.1.6 CommonPublishedInformation

Variable	CommonPublishedInformation	
Description	Common container, aggregated by all modules.It contains published information about vendor and versions.	
Screenshot		
Properties	Property	Value
	Type	Container

3.1.6.1 ArReleaseMajorVersion

Variable	ArReleaseMajorVersion	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label

	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	4

3.1.6.2 ArReleaseMinorVersion

Variable	ArReleaseMinorVersion	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Symbolic Name	6

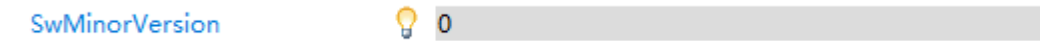
3.1.6.3 ArReleaseRevisionVersion

Variable	ArReleaseRevisionVersion	
Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Symbolic Name	0

3.1.6.4 SwMajorVersion

Variable	SwMajorVersion	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific. Note: Implementation Specific Parameter	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	0

3.1.6.5 SwMinorVersion

Variable	SwMinorVersion	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label

	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	0

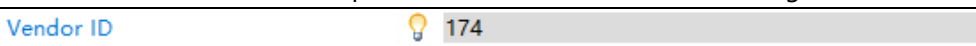
3.1.6.6 SwPatchVersion

Variable	SwPatchVersion	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	0

3.1.6.7 ModuleId

Variable	ModuleId	
Description	Module ID of this module from Module List.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	255

3.1.6.8 VendorId

Variable	VendorId	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	174

Chapter 4 Configuration Guides

4.1 UART Usage Common Steps

Basically, the UART module can be configured by following the below 3 steps:

- 1) Configure UART general configurations and enable UART in Port configuration.
- 2) Configure the DMA required for the UART, if required.
- 3) Generate configuration files.

4.2 UART Channel Demo

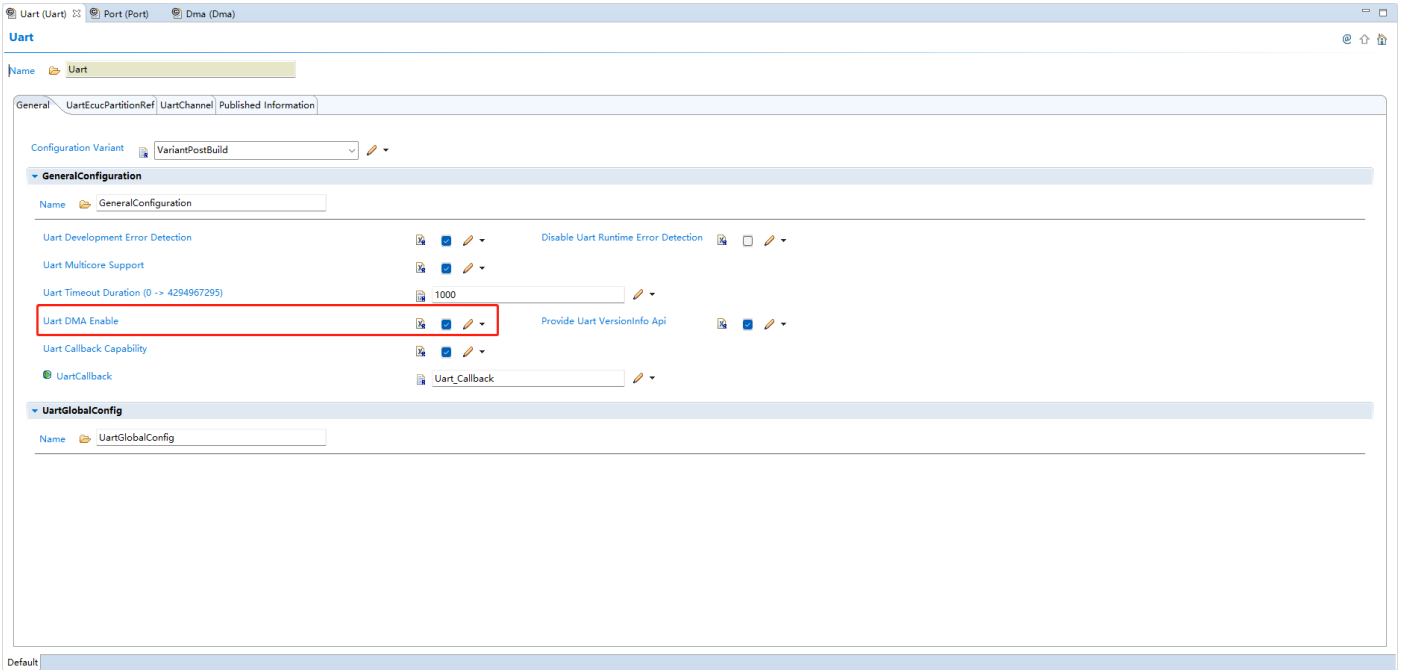
- 1) Set the port pin to UART.

Index	Name	PortPin Id	PortPin PCRN	PortPin Name	PortPin Mode Changeable	PortPin Mode
6	PortPin_6	8	7	PTA7	☒	GPIO
7	PortPin_7	9	8	PTA8	☒	FCUART2_RX
8	PortPin_8	10	9	PTA9	☒	FCUART2_TX
9	PortPin_9	11	10	PTA10	☒	JTAG_TDO
10	KEY3	12	11	PTA11	☒	GPIO
11	PortPin_11	13	12	PTA12	☒	GPIO
12	PortPin_12	14	13	PTA13	☒	GPIO
13	LED3	15	14	PTA14	☒	GPIO
14	PortPin_14	16	15	PTA15	☒	GPIO
15	PortPin_15	17	16	PTA16	☒	GPIO
16	PortPin_16	18	17	PTA17	☒	GPIO
17	PortPin_17	19	18	PTA18	☒	FCUART1_TX
18	PortPin_18	20	19	PTA19	☒	FCUART1_RX
19	PortPin_19	21	20	PTA20	☒	GPIO
20	PortPin_20	22	21	PTA21	☒	FCUART6_RX

- 2) If you need to use the serial port DMA function, configure the corresponding channel in the DMA module

Index	Name	Dma Ch...	Dma Channel Flow Ctrl	Enable P...	Dma Channel Prio...	Enable ...	Enable ...	Enable ...
0	DmaChannelConfig_0	0	DMA_REQ_DISABLED	☒	0	☒	☒	☒
1	DmaChannelConfig_IIC_RX	1	DMA_REQ_FCIC1_RX	☒	1	☒	☒	☒
2	DmaChannelConfig_IIC_TX	2	DMA_REQ_FCIC1_TX	☒	2	☒	☒	☒
3	DmaChannelConfig_OSPI_RX	3	DMA_REQ_OSPI_RX	☒	3	☒	☒	☒
4	DmaChannelConfig_OSPI_TX	4	DMA_REQ_OSPI_TX	☒	4	☒	☒	☒
5	DmaChannelConfig_Can8_Tx	5	DMA_REQ_FLEXCAN8	☒	5	☒	☒	☒
6	DmaChannelConfig_Can8_Rx	6	DMA_REQ_DISABLED	☒	6	☒	☒	☒
7	DmaChannelConfig_FCUART1_RX	7	DMA_REQ_FCUART1_RX	☒	7	☒	☒	☒
8	DmaChannelConfig_FCUART1_TX	8	DMA_REQ_FCUART1_TX	☒	8	☒	☒	☒

The DMA enable in the UART module is as follows:



- 3) Configure the functions of each serial port. The asynchronous mode can choose DMA or interrupt mode. The interrupt service function is configured in UartCallback in the GeneralConfiguration container.

